

Text Manipulation on Linux

09-12-2025

Topics

- **grep**
Link: <https://youtu.be/N05sWPgj-44?si=RqN-cKCBSae4bvK7>
 - **sed**
Link: <https://youtu.be/EACe7aiGczw?si=q8Hyf9d1Vgs0-uj2>
 - **awk**
Link: <https://youtu.be/9Y0ZmI-zWok?si=Larx5zKGgMMSrsn->
 - **find**
Link: <https://youtu.be/BZ5gsFiIK0Q?si=WEDUXPu6DCi2anNL>
 - **Pipes and Redirections (>, », |, <)**
-

Session 4 — Text Manipulation in Linux

Welcome to one of the most *useful and exciting* sessions in your Linux journey. In this session, we learn how Linux becomes a **superpower** when working with text, logs, automation, scripting, and filtering.

These commands are not just "nice-to-have". They are the *backbone* of every serious developer, sysadmin, DevOps engineer, hacker, or sleepy student debugging their broken assignment at 2:30 AM.

By the end of this guide, you will:

- ✓ Search like a pro using **grep**
- ✓ Transform data using **sed**
- ✓ Process structured text using **awk**

- ✓ Locate *anything* using **find**
- ✓ Combine commands like LEGO with **pipes & redirection**

And of course:

- ✓ Solve a bunch of hands-on exercises !!

Let's begin.

1. grep — The Search Engine of the Terminal

Q 1.1 What is grep?

grep stands for **Global Regular Expression Print** — but don't worry, we don't need to understand 1970s developer humor to use it.

In simple words:

grep searches for text inside files or command output.

🧠 1.2 Basic Syntax

```
$ grep [options] "pattern" file
```

Example:

```
$ grep "error" /var/log/syslog
```

Finds all lines containing *error*.

🕒 1.3 Useful Options

-i → ignore case

```
$ grep -i "linux" notes.txt
```

-r → recursive search in directories

```
$ grep -r "password" /etc
```

-n → show line numbers

```
$ grep -n "TODO" project.py
```

-v → invert match (show lines not matching)

```
$ grep -v "DEBUG" logs.txt
```

-E → extended regex

```
$ grep -E "cat|dog" animals.txt
```

1.4 Examples (real and practical)

Find all .pdf files listed in a directory:

```
$ ls | grep ".pdf"
```

Search inside all .log files:

```
$ grep "failed" *.log
```

Search for IP addresses:

```
$ grep -E "[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+" nginx.log
```

🔧 1.5 Exercises

1. **E1.** Find all lines containing the word `error` in `/var/log/syslog`.
2. **E2.** Use `grep` to find lines NOT containing "DEBUG".
3. **E3.** Recursively search for your name in your home directory.

Take your time. Stretch. Drink water. Continue.

2. sed — The Stream Editor

🔧 2.1 What is sed?

`sed` is a **text transformer**.

You can use it to:

- ✓ Replace words
- ✓ Delete lines
- ✓ Insert text
- ✓ Extract text
- ✓ Edit files *without opening them*

🧠 2.2 Basic Syntax

```
$ sed 's/old/new/' file
```

This replaces the *first* occurrence of `old` with `new` per line.

Replace globally (all occurrences):

```
$ sed 's/old/new/g' file
```

✂ 2.3 Common sed operations

Replace text:

```
$ sed 's/linux/Linux/' notes.txt
```

Delete lines containing a word:

```
$ sed '/error/d' logs.txt
```

Print only lines matching a pattern:

```
$ sed -n '/INFO/p' logs.txt
```

Edit a file in-place:

⚠ BE CAREFUL (seriously)

```
$ sed -i 's/foo/bar/g' file.txt
```

💡 2.4 Cool examples

Remove all spaces:

```
$ sed 's/ //g' text.txt
```

Replace multiple spaces with one:

```
$ sed 's/ */ /g'
```

Add prefix to each line:

```
$ sed 's/^/[INFO] /' file.txt
```

🔧 2.5 Exercises

1. **E5.** Replace all 2024 with 2025 in `report.txt`.
2. **E6.** Remove all lines containing the word "temp".
3. **E7.** Add the prefix [LOG] to each line of `output.txt`.
4. **E8.** Delete empty lines.

You're doing great. Take a breath. Continue.

3. awk — The Text Processor King

👑 3.1 What is awk?

awk is a **programming language**... hidden inside a terminal command.
It is perfect for:

- ✓ Extracting columns
- ✓ Summing numbers
- ✓ Processing CSV files
- ✓ Analyzing logs
- ✓ Building mini-scripts

🧠 3.2 Basic Syntax

```
$ awk 'pattern { action }' file
```

Example:

Display first column:

```
$ awk '{print $1}' file.txt
```

🎯 3.3 Field processing

Given a line:

```
Alice 23 Student
```

Fields are:

- \$1 = Alice
- \$2 = 23
- \$3 = Student

🔥 3.4 Useful tricks

Print only the second column:

```
$ awk '{print $2}' data.txt
```

Print lines where second field > 50:

```
$ awk '$2 > 50' data.txt
```

Sum column 3:

```
$ awk '{sum += $3} END {print sum}' sales.txt
```

📖 3.5 Exercises

1. **E9.** Print only usernames from `/etc/passwd`.
2. **E10.** Sum the numbers in the second column of a file.
3. **E11.** Print rows where third column equals "admin".
4. **E12.** Print the first and last field of each line.

You are now officially becoming powerful.

4. find — The Ultimate Search Tool

Q 4.1 What is find?

find searches your filesystem **recursively** for:

- ✓ 📄 Files
- ✓ 📁 Directories
- ✓ 📏 Sizes
- ✓ 🔒 Permissions
- ✓ 🔍 Names
- ✓ 📄 Types
- ✓ Everything

🧠 4.2 Basic Syntax

```
$ find path options
```

Example:

```
$ find . -name "*.txt"
```

🎯 4.3 Essential Options

Search by name:

```
$ find /home -name "notes.txt"
```

Case-insensitive:

```
$ find /home -iname "linux"
```


Search for directories:

```
$ find . -type d -name "src"
```

Search by size:

```
$ find . -size +1M
```

Search by permissions:

```
$ find / -perm 777
```

Exclude matching:

```
$ find . ! -name "*.txt"
```

☰ 4.4 Exercises

1. **E13.** Find all .py files in your home directory.
2. **E14.** Find all files larger than 10 MB.
3. **E15.** List all directories named backup.
4. **E16.** Find files with permission 777.

Great! ☕ Grab some coffee. The final boss fight is next.

5. Pipes and Redirection

🔧 5.1 What are pipes?

Pipes (|) send the output of one command into another.

Think of it as plugging commands together like LEGO Pieces.

Example:

```
$ ls | grep ".txt"
```

→ 5.2 Output Redirection

Write to a file:

```
$ echo "Hello" > file.txt
```

Append to a file:

```
$ echo "World" >> file.txt
```

Redirect command output:

```
$ ls > list.txt
```

← 5.3 Input Redirection

```
$ command < file
```

Example:

```
$ wc -l < notes.txt
```

🔗 5.4 Combining pipes

Count python files:

```
$ ls | grep ".py" | wc -l
```

Get top 10 largest files:

```
$ du -ah | sort -h | tail
```

See unique words (sorted):

```
$ cat text.txt | tr ' ' '\n' | sort | uniq
```

🔧 5.5 Exercises

1. **E17.** Count how many lines contain "error" in `logs.txt`.
 2. **E18.** Save all `.txt` filenames into `files.txt`.
 3. **E19.** Print the first 5 unique words in `story.txt`.
 4. **E20.** Display only lines containing "user" from `/etc/passwd`.
-

🏆 Conclusion

Congratulations — you just learned the *core skills* that make Linux insanely powerful.
From now on:

- Logs won't scare you
- Large files won't slow you
- Searching becomes instant
- Processing data becomes easy
- Automation becomes natural
- Your terminal becomes a **superweapon**

This session prepares you for scripting, DevOps, automation, cybersecurity, and everything cool in Linux.